https://algs4.cs.princeton.edu

2.3 QUICKSORT

- *quicksort*
- *selection*
- *duplicate keys*
- *system sorts*

Two classic sorting algorithms: mergesort and quicksort

Critical components in the world's computational infrastructure.

- Full scientific understanding of their properties has enabled us to develop them into practical system sorts.
- Quicksort honored as one of top 10 algorithms of 20th century in science and engineering.

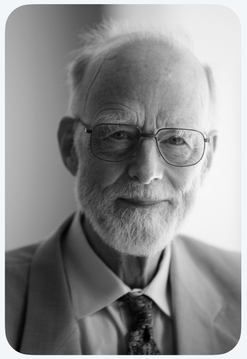
Mergesort. [last lecture]

Quicksort. [this lecture]

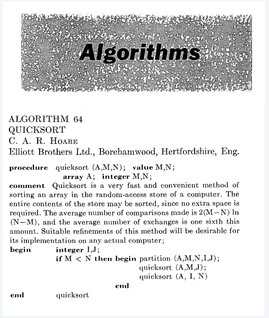
A brief history

Tony Hoare.

- Invented quicksort in 1960 to translate Russian into English.
- Later learned Algol 60 (and recursion) to implement it.



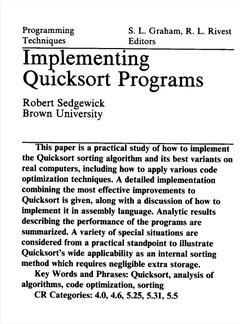
Tony Hoare
1980 Turing Award



Algorithms

ALGORITHM 64
QUICKSORT
C. A. R. HOARE
Elliott Brothers Ltd., Borehamwood, Hertfordshire, Eng.

```
procedure  quicksort (A,M,N); value M,N;
           array A; integer M,N;
comment    Quicksort is a very fast and convenient method of
           sorting an array in the random-access store of a computer. The
           entire contents of the store may be sorted, since no extra space is
           required. The average number of comparisons made is 2(M-N) ln
           (N-M), and the average number of exchanges is one sixth this
           amount. Suitable refinements of this method will be desirable for
           its implementation on any actual computer;
begin      integer I,J;
           if M < N then begin partition (A,M,N,I,J);
                              quicksort (A,M,J);
                              quicksort (A, I, N)
                         end
end        quicksort
```



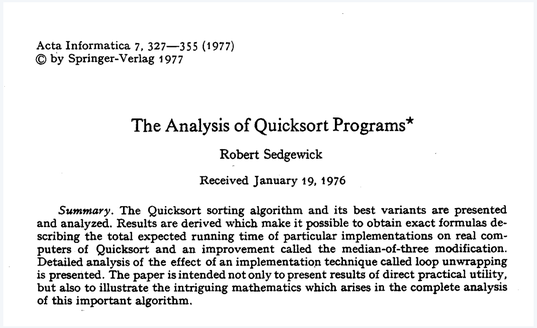
Programming Techniques — S. L. Graham, R. L. Rivest Editors

Implementing Quicksort Programs

Robert Sedgewick
Brown University

This paper is a practical study of how to implement the Quicksort sorting algorithm and its best variants on real computers, including how to apply various code optimization techniques. A detailed implementation combining the most effective improvements to Quicksort is given, along with a discussion of how to implement it in assembly language. Analytic results describing the performance of the programs are summarized. A variety of special situations are considered from a practical standpoint to illustrate Quicksort's wide applicability as an internal sorting method which requires negligible extra storage.

Key Words and Phrases: Quicksort, analysis of algorithms, code optimization, sorting
CR Categories: 4.0, 4.6, 5.25, 5.31, 5.5



Acta Informatica 7, 327—355 (1977)

The Analysis of Quicksort Programs*

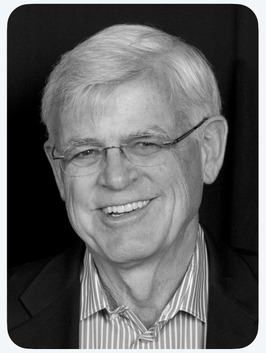
Robert Sedgewick

Received January 19, 1976

Summary. The Quicksort sorting algorithm and its best variants are presented and analyzed. Results are derived which make it possible to obtain exact formulas describing the total expected running time of particular implementations on real computers of Quicksort and an improvement called the median-of-three modification. Detailed analysis of the effect of an implementation technique called loop unwrapping is presented. The paper is intended not only to present results of direct practical utility, but also to illustrate the intriguing mathematics which arises in the complete analysis of this important algorithm.

Bob Sedgewick.

- Refined and popularized quicksort in 1970s.
- Analyzed many versions of quicksort.



Bob Sedgewick

https://algs4.cs.princeton.edu

2.3 QUICKSORT

- *quicksort*
- *selection*
- *duplicate keys*
- *system sorts*

Quicksort overview

shuffle的目的在于，我们希望每次划分得到的两个子数组的sz尽量相等，shuffle可以尽可能避免出现0和n-1长度的两个子数组

Step 1. Shuffle the array.

Step 2. Partition the array so that, for some index `j` :

- Entry `a[j]` is in place. ⟵ *"pivot" or "partitioning item"*
- No larger entry to the left of `j`.
- No smaller entry to the right of `j`.

Step 3. Sort each subarray recursively.

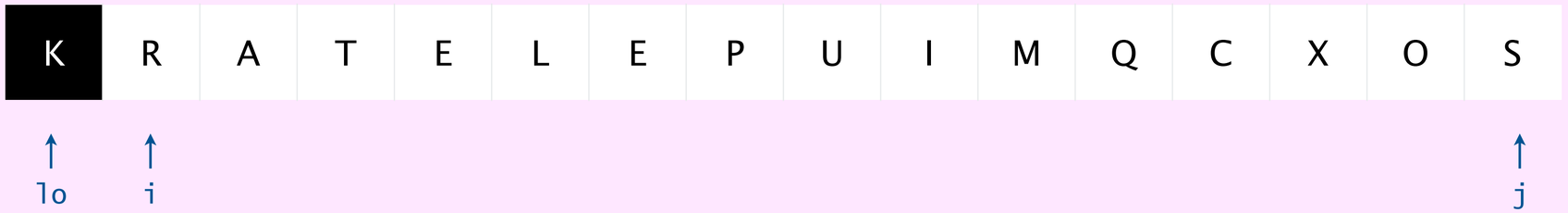
分区的过程不是猜位置，是计算位置。
分区的工作方式是：【伪代码见下页】
用双指针从左右往中间扫
把比基准小的换到左边
把比基准大的换到右边
最后把基准交换到左右两部分的中间
这个过程不是概率问题，是确定性步骤。

总体上看，依然是每次都找一个中间位置，划分出两个子数组，然后对子数组分别进行上面的操作

Quicksort partitioning demo

Repeat until pointers cross:

- Scan `i` from left to right so long as `a[i] < a[lo]`.
- Scan `j` from right to left so long as `a[j] > a[lo]`.
- Exchange `a[i]` with `a[j]`.



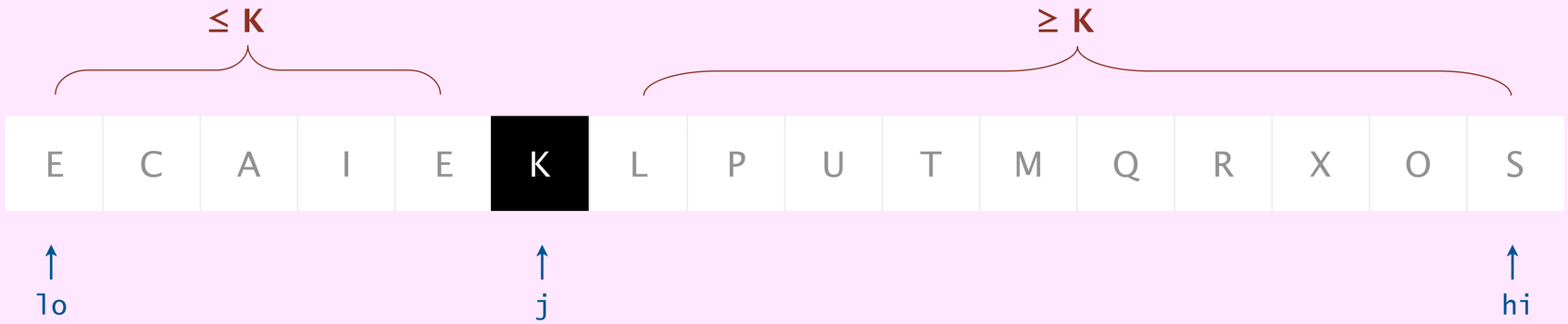
stop i scan because a[i] >= a[lo]

Quicksort partitioning demo

Repeat until pointers cross:

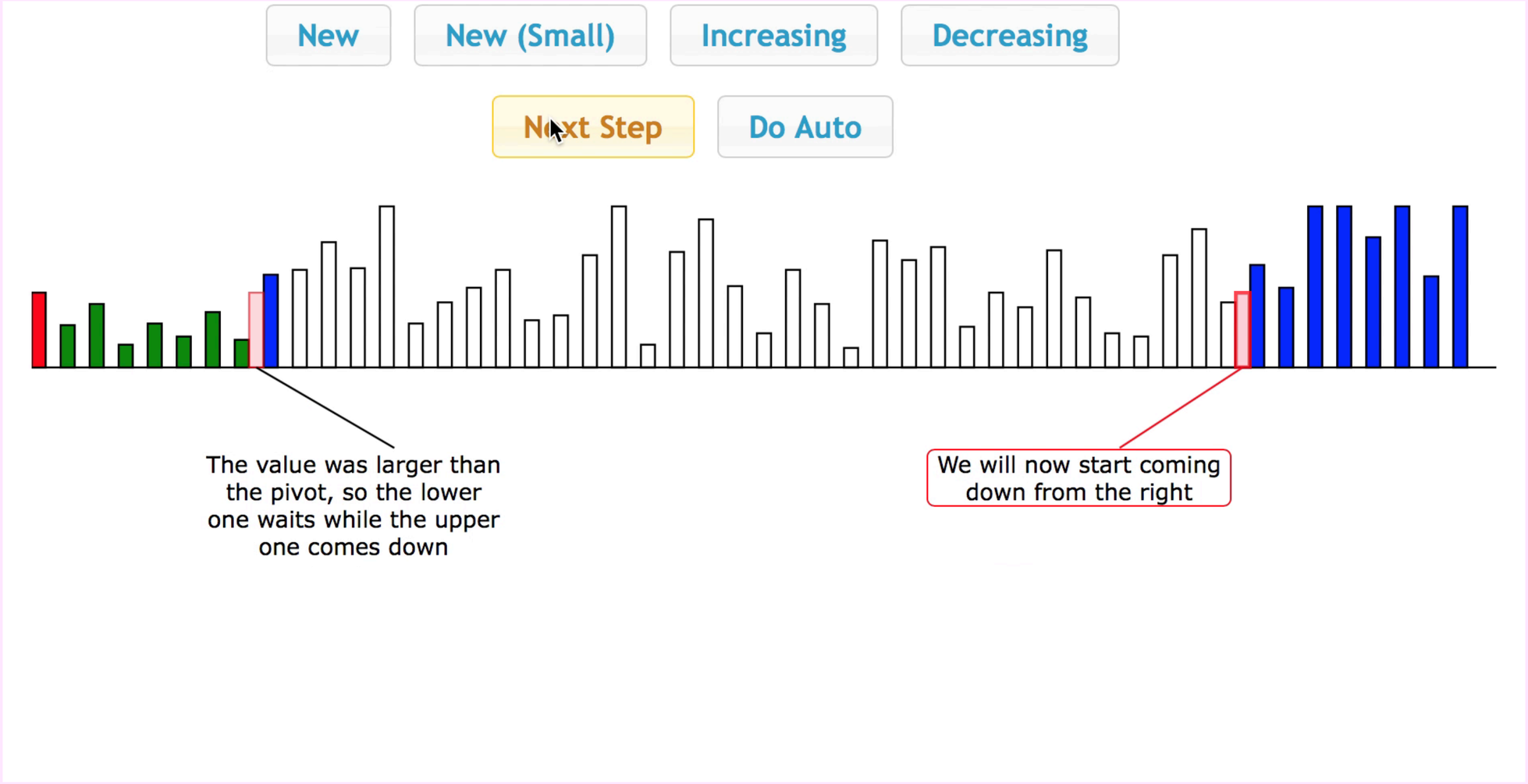
- Scan `i` from left to right so long as `a[i] < a[lo]`.
- Scan `j` from right to left so long as `a[j] > a[lo]`.
- Exchange `a[i]` with `a[j]`.

When pointers cross. Exchange `a[lo]` with `a[j]`.



partitioned!

The music of quicksort partitioning (by Brad Lyon)



https://learnforeverlearn.com/pivot_music

Quicksort partitioning: Java implementation

这种排序不需要辅助数组（对比mergesort：会new一个新数组）

```
private static int partition(Comparable[] a, int lo, int hi) {
   Comparable p = a[lo];
   int i = lo, j = hi+1;
   while (true) {

      while (less(a[++i], p))
         if (i == hi) break;

      while (less(p, a[--j]))
         if (j == lo) break;

      if (i >= j) break;

      exch(a, i, j);
   }

   exch(a, lo, j);
   return j;
}
```

find item on left to swap

find item on right to swap

check if pointers cross

swap

注意特点：j原来一定是小于等于K的

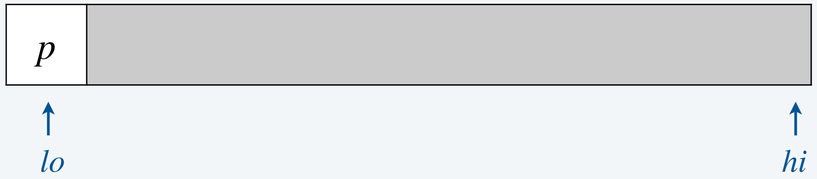
swap with pivot

index of element known to be in place

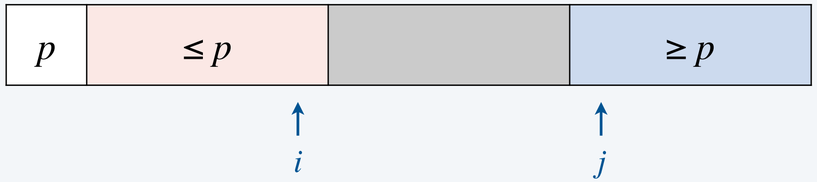
返回交换后，K所在的位置

https://algs4.cs.princeton.edu/23quick/Quick.java.html

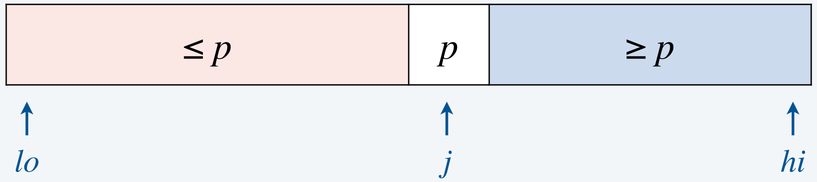
before



during



after



Quicksort: quiz 2

In the worst case, how many compares and exchanges does `partition()` make to partition a subarray of length n?

一个问题包含了两个变量

A. $\sim \frac{1}{2} n$ and $\sim \frac{1}{2} n$

B. $\sim \frac{1}{2} n$ and $\sim n$

C. $\sim n$ and $\sim \frac{1}{2} n$

D. $\sim n$ and $\sim n$

我觉得应该是C

要注意，在一开始的排序中，是下标为i和j的进行交换，不是和K进行交换

交换的最坏情况，是每次i和j都要交换，因此是n/2次

M	A	B	C	D	E	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10

Quicksort: Java implementation

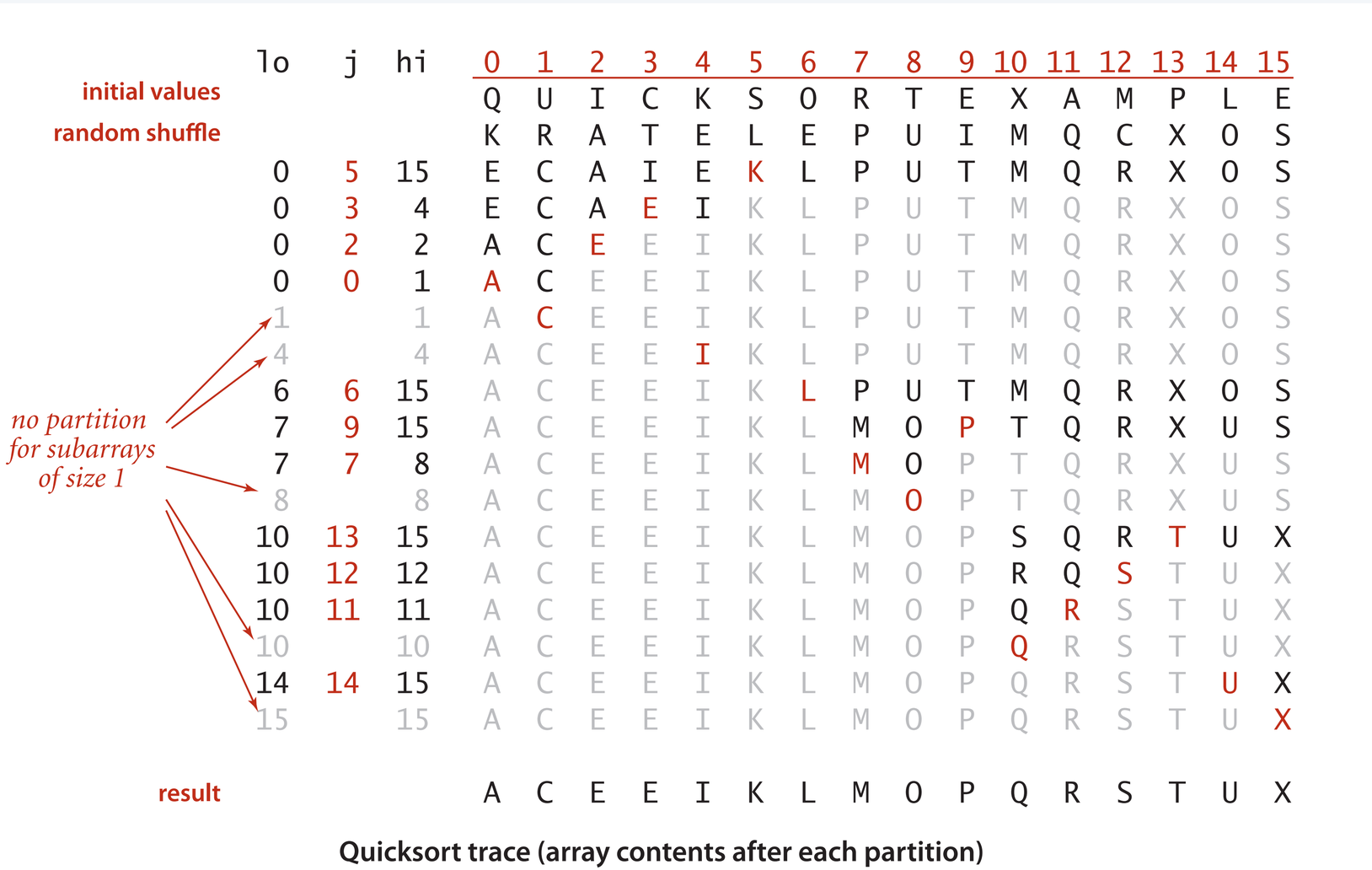
```
public class Quick {

   private static int partition(Comparable[] a, int lo, int hi) {
      /* see previous slide */
   }

   public static void sort(Comparable[] a) {
      StdRandom.shuffle(a);
      sort(a, 0, a.length - 1);
   }

   private static void sort(Comparable[] a, int lo, int hi) {
      if (hi <= lo) return;
      int j = partition(a, lo, hi);
      sort(a, lo, j-1);
      sort(a, j+1, hi);
   }

}
```

shuffle needed for performance guarantee (stay tuned)

两个 sort 方法是同名不同参数的方法重载

https://algs4.cs.princeton.edu/23quick/Quick.java.html

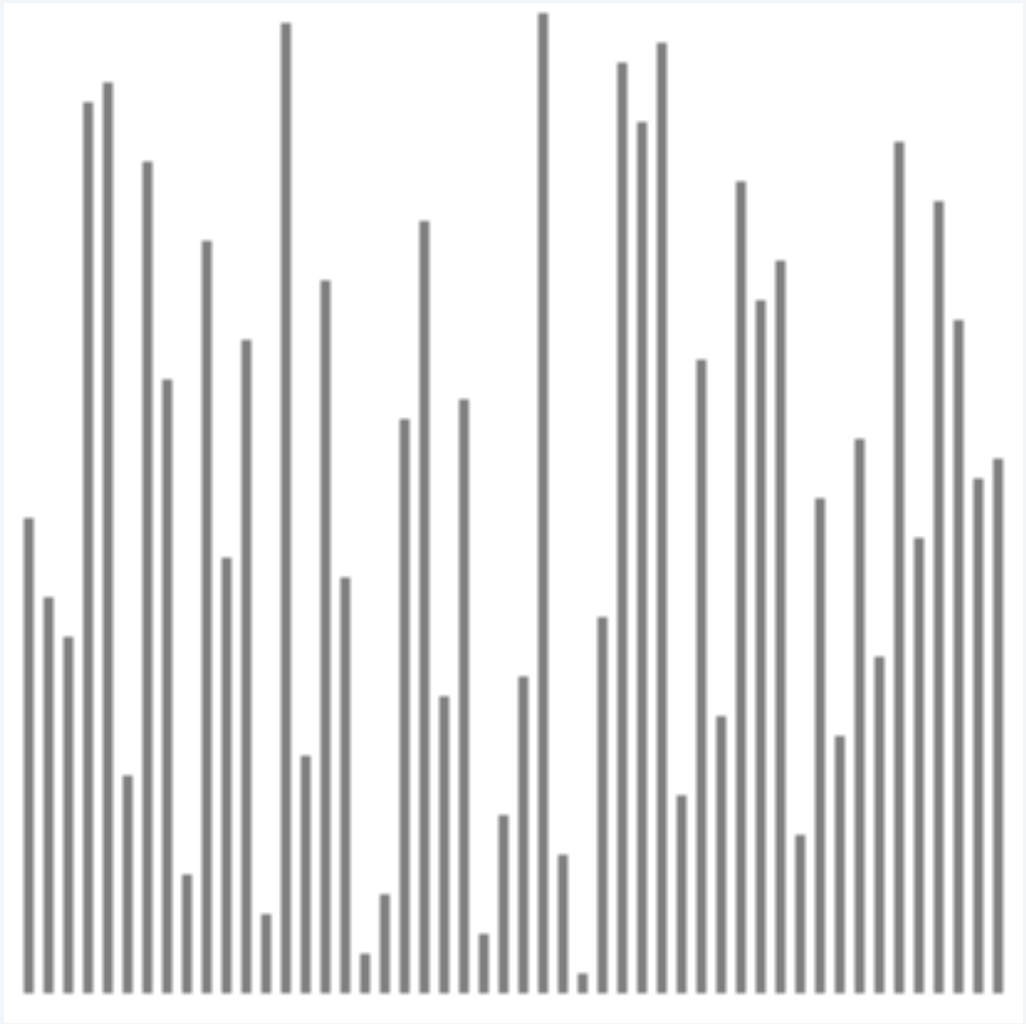
Quicksort trace



Quicksort trace (array contents after each partition)

Quicksort animation

50 random items



algorithm position

in order

current subarray

not in order

https://www.toptal.com/developers/sorting-algorithms/quick-sort

Quicksort: implementation details

Partitioning in-place. Using an extra array makes partitioning easier (and stable), but it is not worth the cost.

Loop termination. Terminating the loop (when pointers cross) is more subtle than it appears.

Equal keys. Handling duplicate keys is trickier that it appears. [stay tuned]

Preserving randomness. Shuffling is needed for performance guarantee.

Equivalent alternative. Pick a random pivot in each subarray.

Quicksort: empirical analysis

Running time estimates:

- Home PC executes 10^8 compares/second.
- Supercomputer executes 10^{12} compares/second.

	insertion sort (n^2)			mergesort (n log n)			quicksort (n log n)		
computer	thousand	million	billion	thousand	million	billion	thousand	million	billion
home	*instant*	2.8 hours	317 years	*instant*	1 second	18 min	*instant*	0.6 sec	12 min
super	*instant*	1 second	1 week	*instant*	*instant*	*instant*	*instant*	*instant*	*instant*

Lesson 1. Good algorithms are better than supercomputers.

Lesson 2. Great algorithms are better than good ones.

Quicksort: quiz 3

在下面用Cn分析得到理论比较次数1.39nlogn
并且MS在pdf中也有算出比较次数是nlogn，所以理论上QS的比较次数是要更多的

Why is quicksort typically faster than mergesort in practice?

A. Fewer compares.

B. Fewer array acceses.

C. Both A and B.

D. Neither A nor B.

quick：nlogn（最坏n方，一般随机打乱之后不会出现），nlogn
merge：nlogn，说明增长趋势一样，但是实际使用中的区别就在于“常数因子”，可以理解为nlogn的系数
归并必须两两比较，几乎没有捷径
快排是划分（partition），一次比较就能确定一个元素的最终位置，总比较次数少

快速排序：原地分区，仅在原数组上交换元素，缓存命中率高，访问开销小
归并排序：需要额外数组（或递归栈 + 临时空间），合并时存在大量数据拷贝，数组访问次数更多【元素移动次数也变多】

元素比较，移动，交换次数【算法落地时面临的三大挑战】

Quicksort: worst-case analysis

Worst case. Number of compares is $\sim \frac{1}{2} n^2$.

			a[ ]														
lo	j	hi	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
0	0	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	1	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
2	2	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
3	3	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
4	4	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	5	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
6	6	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
7	7	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
8	8	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
9	9	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
10	10	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
11	11	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
12	12	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
13	13	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
14		14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O

after random shuffle

Quicksort: worst-case analysis

Worst case. Number of compares is $\sim \frac{1}{2} n^2$.

a[ ]

lo	j	hi	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O

← *after random shuffle*

Good news. Worst case for randomized quicksort is mostly irrelevant in practice.

- Exponentially small chance of occurring.
  (unless bug in shuffling or no shuffling)
- More likely that computer is struck by lightning bolt during execution.

随机打乱数组或随机选择 pivot 后，出现最坏划分的概率呈指数级减小。

Quicksort: probabilistic analysis

Proposition. The expected number of compares C_n to quicksort an array of n distinct keys is $\sim 2n \ln n$ (and the number of exchanges is $\sim \frac{1}{3} n \ln n$).

Recall. Any algorithm with the following structure takes $\Theta(n \log n)$ time.

```
public static void f(int n) {
    if (n == 0) return;
    f(n/2);
    f(n/2);
    linear(n);
}
```

solve two problems of half the size

do $\Theta(n)$ work

Intuition. Each partitioning step divides the problem into two subproblems, each of approximately one-half the size.

probabilistically "close enough"

Quicksort: probabilistic analysis

Proposition. The expected number of compares C_n to quicksort an array of n distinct keys is $\sim 2n \ln n$ (and the number of exchanges is $\sim \frac{1}{3} n \ln n$).

Cn意义：元素个数为n时，需要进行的比较次数

递归的“自相似”性：递归的表达，可以用同样的符号

Pf. C_n satisfies the recurrence $C_0 = C_1 = 0$ and for $n \geq 2$:

pivot和n-1个元素的比较+边界比较

partitioning

left *right*

随机地划分左右子数组，共n种，因此每个是1/n的概率

$$C_n = (n+1) + \left(\frac{C_0 + C_{n-1}}{n}\right) + \left(\frac{C_1 + C_{n-2}}{n}\right) + \ldots + \left(\frac{C_{n-1} + C_0}{n}\right)$$

partitioning probability

- Multiply both sides by n and collect terms:

$$n\,C_n = n(n+1) + 2(C_0 + C_1 + \ldots + C_{n-1})$$

- Subtract from this equation the same equation for $n - 1$: 先展开（n-1）Cn-1，然后用1式子减去展开式子得到下面的

$$n\,C_n - (n-1)\,C_{n-1} = 2n + 2\,C_{n-1}$$

- Rearrange terms and divide by $n\,(n+1)$:

$$\frac{C_n}{n+1} = \frac{C_{n-1}}{n} + \frac{2}{n+1}$$

analysis beyond scope of this course

Quicksort: probabilistic analysis

- Repeatedly apply previous equation:

$$\frac{C_n}{n+1} = \frac{C_{n-1}}{n} + \frac{2}{n+1}$$

展开

$$= \frac{C_{n-2}}{n-1} + \frac{2}{n} + \frac{2}{n+1} \quad \longleftarrow \textit{substitute previous equation}$$

$$= \frac{C_{n-3}}{n-2} + \frac{2}{n-1} + \frac{2}{n} + \frac{2}{n+1}$$

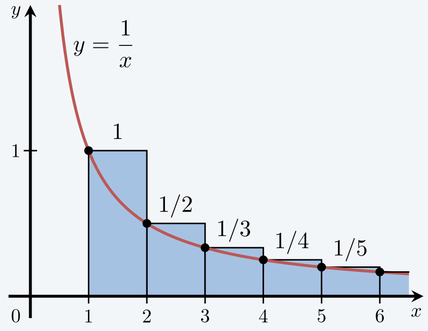
$$= \frac{2}{3} + \frac{2}{4} + \frac{2}{5} + \ldots + \frac{2}{n+1}$$

- Approximate sum by an integral: integral：积分

$$C_n = 2(n+1)\left(\frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \ldots + \frac{1}{n+1}\right)$$

$$\sim 2(n+1)\int_3^{n+1} \frac{1}{x}\,dx$$

看作是底为1，高为1/n的积分



- Finally, the desired result:

$$C_n \sim 2(n+1)\ln n \approx 1.39\, n \lg n$$

Quicksort properties

Quicksort analysis summary.

- Expected number of compares is $\sim 1.39\, n \log_2 n$. ← *39% more than mergesort*

  [standard deviation is $\sim 0.65\, n$]

  但是有更少的元素移动

- Expected number of exchanges is $\sim 0.23\, n \log_2 n$. ← *much less than mergesort*
- Min number of compares is $\sim\ n \log_2 n$. ← *never less than mergesort*
- Max number of compares is $\sim \frac{1}{2}\, n^2$. ← *but never happens*

Context. Quicksort is a (Las Vegas) randomized algorithm.

- Guaranteed to be correct.
- Running time depends on outcomes of random coin flips (shuffle).

Quicksort properties

原位排序：
如果使用的额外空间在O(logn)或者更少，栈空间可计入logn
所以QSort是原位排序（常量的空间需求），但MergeSort不是（nlogn）

Proposition. Quicksort is an in-place sorting algorithm.

- Partitioning: $\Theta(1)$ extra space.
- Function-call stack: $\Theta(\log n)$ extra space (with high probability).

can guarantee $\Theta(\log n)$ depth by recurring on smaller subarray before larger subarray (but this involves using an explicit stack)

栈空间：每次递归调用都会在调用栈中压入一个栈帧（stack frame），用来保存当前函数的参数、局部变量、返回地址等信息。
每个栈帧的大小是常数（与输入规模 n 无关）。
因此，总栈空间 ≈ 递归深度 × 单个栈帧的常数开销。

Proposition. Quicksort is not stable.

Pf. [by counterexample] 反例

i	j	0	1	2	3
		B_1	C_1	C_2	A_1
1	3	B_1	C_1	C_2	A_1
1	3	B_1	A_1	C_2	C_1
0	1	A_1	B_1	C_2	C_1

stable：稳定排序：排序后，值相等的元素会保持它们在原数组中的相对顺序。
MS是，但QS不是，比如左边，C1和C2都是相等元素C，原来C1在左边，但是一换后，C1跑到右边去了

Quicksort: practical improvements

Insertion sort small subarrays.

- Even quicksort has too much overhead for tiny subarrays.
- Cutoff to insertion sort for ≈ 10 items.

引入插入排序处理小子串：
原因：快速排序的递归调用、分区操作在小数组上的固定开销会盖过其平均复杂度优势，而插入排序在小规模数据上（元素交换少、无递归）反而更快。
解决：在长度低于某个值的时候用插入排序

```
private static void sort(Comparable[] a, int lo, int hi) {

   if (hi <= lo + CUTOFF - 1) {
      Insertion.sort(a, lo, hi);
      return;
   }

   int j = partition(a, lo, hi);
   sort(a, lo, j-1);
   sort(a, j+1, hi);
}
```

Quicksort: practical improvements

Median of sample.

- Best choice of pivot item = median.
- Estimate true median by taking median of sample.
- Median-of-3 (random) items.

~ 12/7 $n \ln n$ *compares (14% fewer)*
~ 12/35 $n \ln n$ *exchanges (3% more)*

```
private static void sort(Comparable[] a, int lo, int hi) {
   if (hi <= lo) return;

   int median = medianOf3(a, lo, mid + (hi - lo) / 2, hi);
   swap(a, lo, median);

   int j = partition(a, lo, hi);
   sort(a, lo, j-1);
   sort(a, j+1, hi);
}
```

返回值 j 是基准元素最终的位置

取三者的中位数，作为基准，中位数不是平均数，中位数一定是其中的一个值，返回中位数所在的位置

```
private static int medianOf3(Comparable[] a, int lo, int mid, int hi)
{
    // 比较三个元素，返回中间大小元素的索引
    if (less(a[mid], a[lo])) swap(a, lo, mid);   // 保证 a[lo] <= a
[mid]
    if (less(a[hi], a[lo])) swap(a, lo, hi);     // 保证 a[lo] <= a[hi
], 此时 a[lo] 是三者最小值
    if (less(a[hi], a[mid])) swap(a, mid, hi);   // 保证 a[mid] <= a
[hi], 此时 a[mid] 是中位数
    return mid; // 中位数已被交换到 mid 位置
}
```

https://algs4.cs.princeton.edu

2.3 QUICKSORT

- *quicksort*
- *selection*
- *duplicate keys*
- *system sorts*

Selection

Goal. Given an array of n items, find item of rank k.

Ex. Min $(k = 0)$, max $(k = n - 1)$, median $(k = n / 2)$.

Applications.

- Order statistics.
- Find the "top k." 找到第k大的那个元素

Use complexity theory as a guide.

- Easy $O(n \log n)$ algorithm. How? 先排序，再遍历一轮
- Easy $O(n)$ algorithm for $k = 0$ or 1. How? 扫一遍，维护最小的那两个
- Easy $\Omega(n)$ lower bound. Why? 下界：对于任意算法，至少每个元素都要看一遍

Which is true?

- $O(n)$ algorithm? [is there a linear-time algorithm?] 对于任意k都存在O(n)算法？
- $\Omega(n \log n)$ lower bound? [is selection as hard as sorting?]

Quickselect demo

Partition array so that for some `j`:

- Entry `a[j]` is in place.
- No larger entry to the left of `j`.
- No smaller entry to the right of `j`.

Repeat in one subarray, depending on `j`; stop when `j` equals `k`.

select element of rank k = 5

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
50	21	28	65	39	59	56	22	95	12	90	53	32	77	33

k = 5

Quickselect

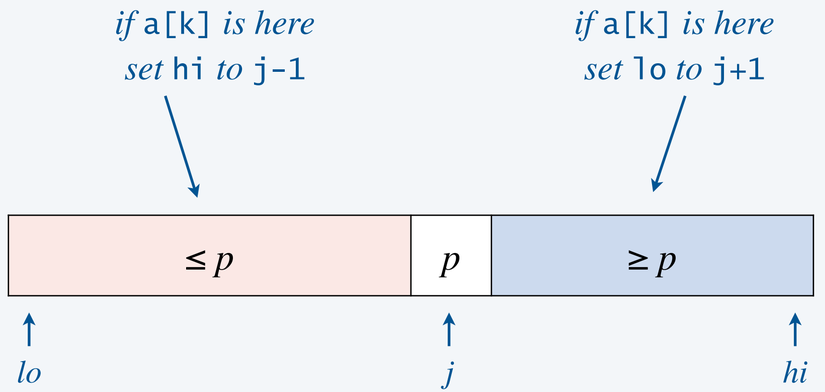
Partition array so that for some `j`:

- Entry `a[j]` is in place.
- No larger entry to the left of `j`.
- No smaller entry to the right of `j`.

基于快排的思想，先找中间元素的排序j，看j和k的大小关系如果k＞j，那么只需要在右子串中查找

Repeat in one subarray, depending on `j`; stop when `j` equals `k`.

```
public static Comparable select(Comparable[] a, int k) {
    StdRandom.shuffle(a);
    int lo = 0, hi = a.length - 1;
    while (hi > lo) {
       int j = partition(a, lo, hi);
       if      (j < k) lo = j + 1;
       else if (j > k) hi = j - 1;
       else            return a[k];
    }
    return a[k];
}
```



Quickselect: probabilistic analysis

Proposition. The expected number of compares C_n to quickselect the item of rank k in an array of length n is $\Theta(n)$.

probabilistically "close enough"

Intuition. Each partitioning step approximately halves the length of the array.

Recall. Any algorithm with the following structure takes $\Theta(n)$ time.

学习抽象方式，f(n)表示输入大小为n的开销

```
public static void f(int n) {
   if (n == 0) return;
   linear(n);      ←—— do Θ(n) work   划分所需要的开销
   f(n/2);         ←—— solve one subproblem of half the size
}
```

$$n + n/2 + n/4 + \ldots + 1 \sim 2n$$

Careful analysis yields:

$$C_n \sim 2n + 2k \ln(n/k) + 2(n-k) \ln(n/(n-k))$$

$$\leq (2 + 2 \ln 2)\, n$$

$$\approx 3.38\, n$$

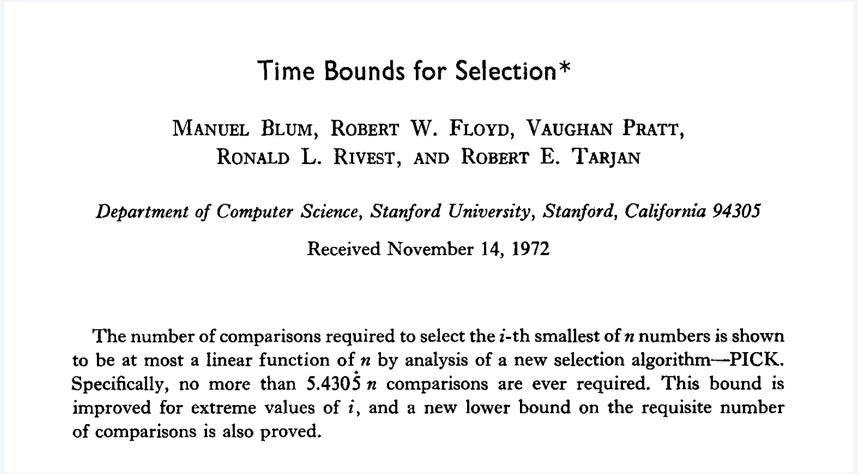
max occurs for median ($k = n/2$)

平均情况下的期望开销（最开始做了一个shuffle）

Theoretical context for selection

Q. Compare-based selection algorithm that makes $\Theta(n)$ compares in the worst case?

A. Yes! [ingenious divide-and-conquer]



Time Bounds for Selection*

MANUEL BLUM, ROBERT W. FLOYD, VAUGHAN PRATT,
RONALD L. RIVEST, AND ROBERT E. TARJAN

Department of Computer Science, Stanford University, Stanford, California 94305

Received November 14, 1972

The number of comparisons required to select the i-th smallest of n numbers is shown to be at most a linear function of n by analysis of a new selection algorithm—PICK. Specifically, no more than $5.430\overline{5}\,n$ comparisons are ever required. This bound is improved for extreme values of i, and a new lower bound on the requisite number of comparisons is also proved.

$$T(n) = T(n/5) + T(7n/10) + \Theta(n)$$

find pivot — *that eliminates 30% of items*

Caveat. Constants are high $\Rightarrow$ not used in practice.

Use theory as a guide.

- Open problem: practical algorithm that makes $\Theta(n)$ compares in the worst case.
- Until one is discovered, use quickselect (if you don't need a full sort).

https://algs4.cs.princeton.edu

2.3 QUICKSORT

- *quicksort*
- *selection*
- *duplicate keys*
- *system sorts*

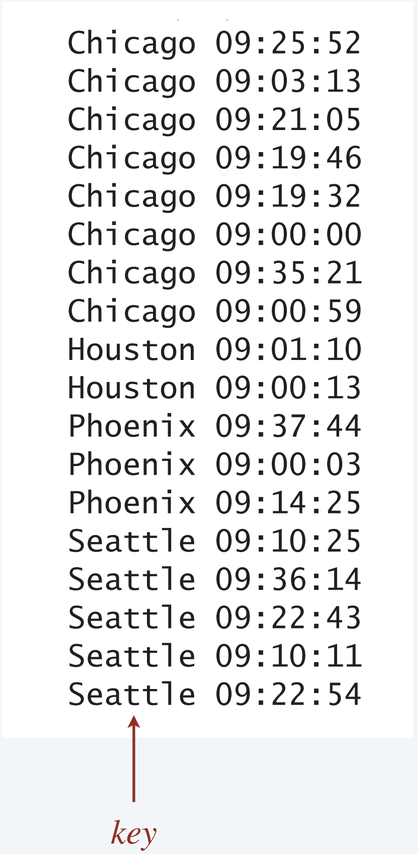
Duplicate keys

Often, purpose of sort is to bring items with equal keys together.

- Sort population by age.
- Remove duplicates from mailing list.
- Sort job applicants by college attended.

Typical characteristics of such applications.

- Huge array.
- Small number of key values.



Quicksort: quiz 4

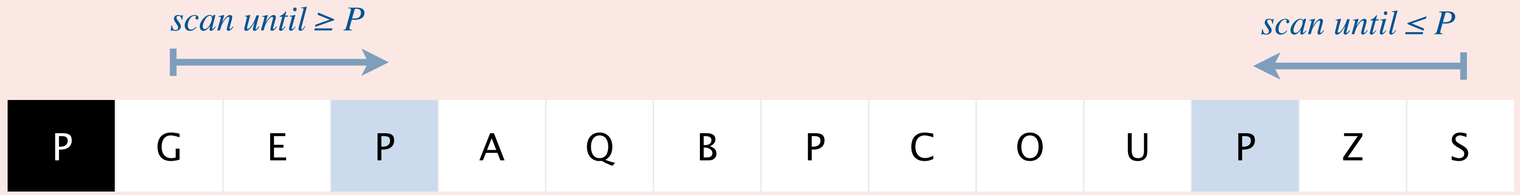
When partitioning, how to handle keys equal to pivot?

A会导致极端情况，比如所有元素都相等的情况下，按照扫描规则，从左会先一直扫描到尾部，导致子串长度是n-1和0，导致最坏情况变成n方复杂度

A.

B正确：将等于 P 的元素均匀分到左右子数组，保证分区平衡，避免重复元素导致的性能退化

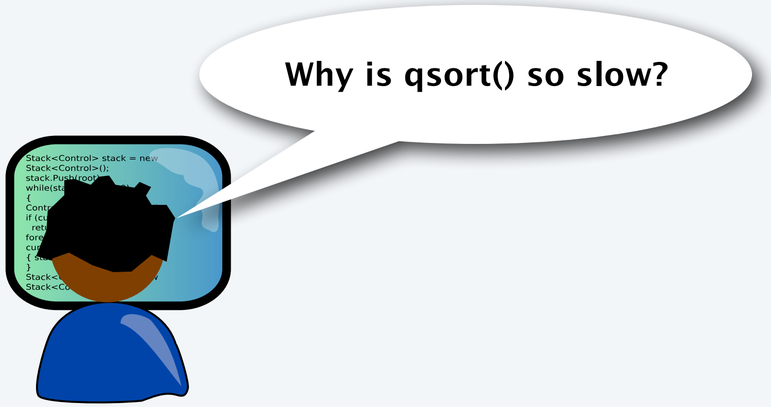
B.

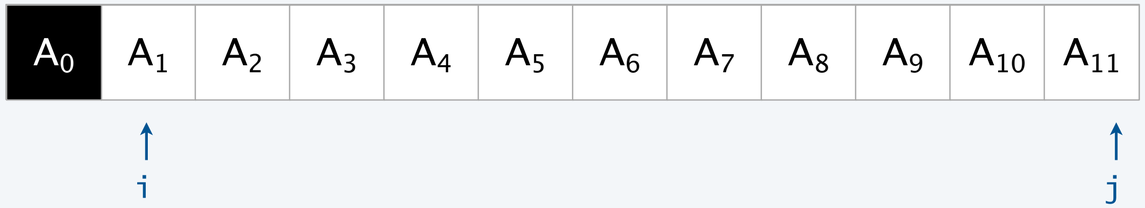


C. Either A or B.

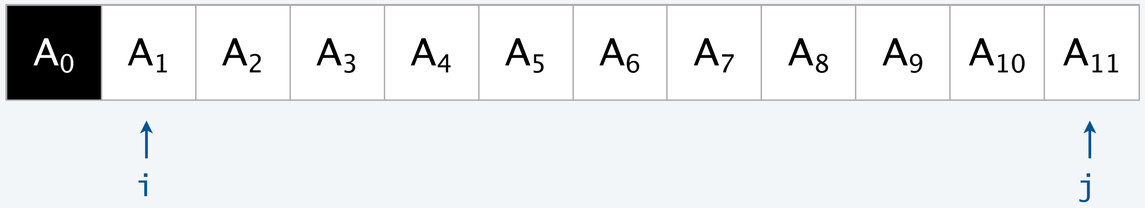
War story (system sort in C)

Bug. A `qsort()` call in C that should have taken seconds was taking minutes to sort a random array of 0s and 1s.





skip over equal keys



stop scan on equal keys

Duplicate keys: partitioning strategies

Bad. Don't stop scans on equal keys.

[$\Theta(n^2)$ compares when all keys equal]

遇到等于基准（pivot）的元素不停止扫描，会导致重复元素被反复交换、分区失衡。
当所有键相等时，时间复杂度退化为 Θ(n²)（最坏情况）

Good. Stop scans on equal keys.

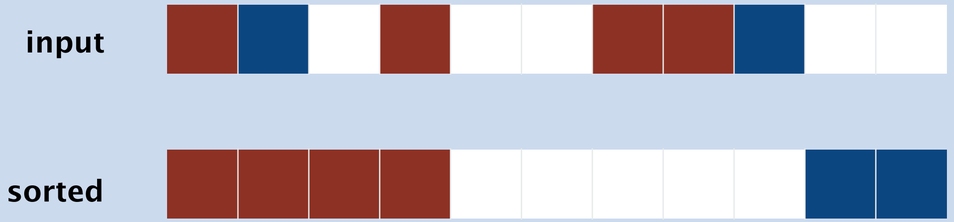
[$\sim n \log_2 n$ compares when all keys equal]

Better. Put all equal keys in place. How?

[$\sim n$ compares when all keys equal]

Dutch National Flag Problem

Problem. [Edsger Dijkstra] Given an array of n buckets, each containing a red, white, or blue pebble, sort them by color.



Operations allowed.

- $swap(i, j)$: swap the pebble in bucket i with the pebble in bucket j.
- $getColor(i)$: determine the color of the pebble in bucket i.

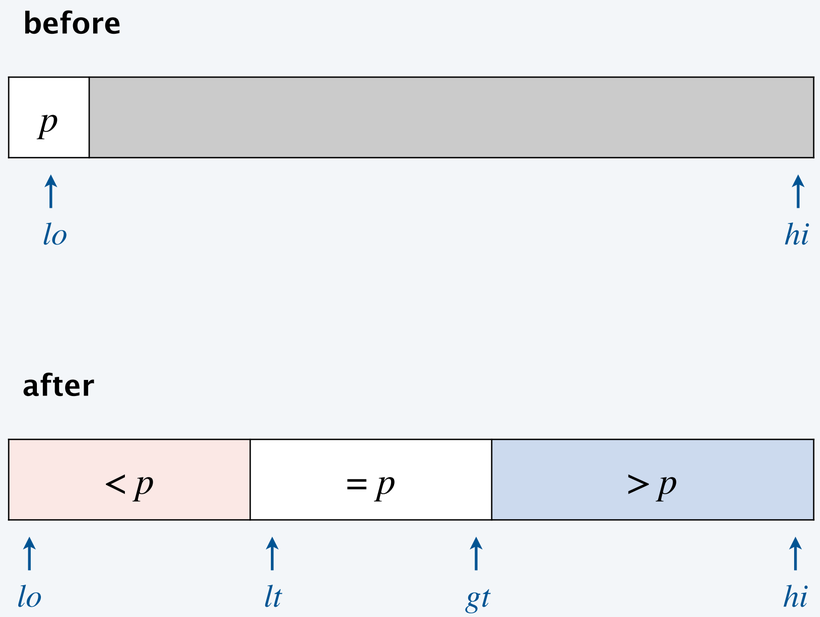
Performance requirements.

- Exactly n calls to $getColor()$.
- At most n calls to $swap()$.
- $\Theta(1)$ extra space.

3-way partitioning

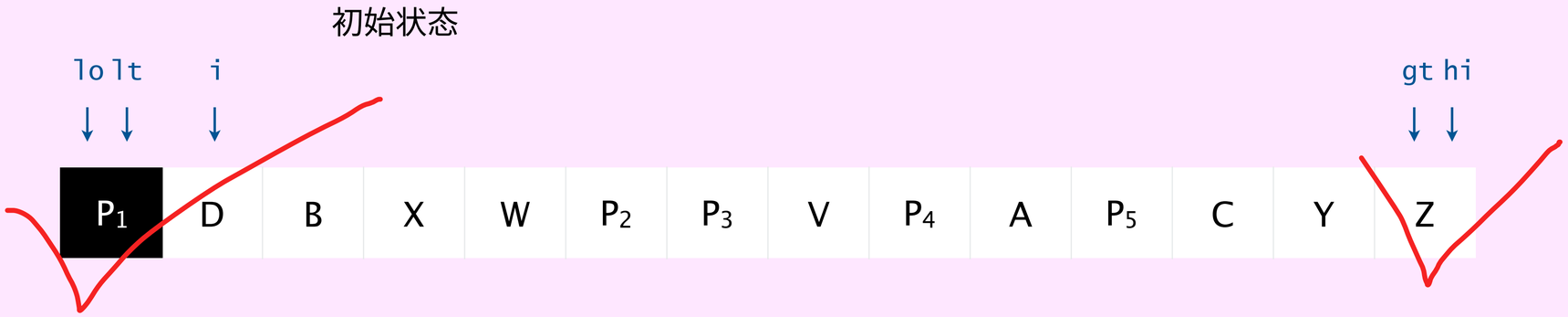
Goal. Use pivot `p = a[lo]` to partition array into three parts so that:

- Red: smaller entries to the left of `lt`.
- White: equal entries between `lt` and `gt`.
- Blue: larger entries to the right of `gt`.



Dijkstra's 3-way partitioning algorithm: demo

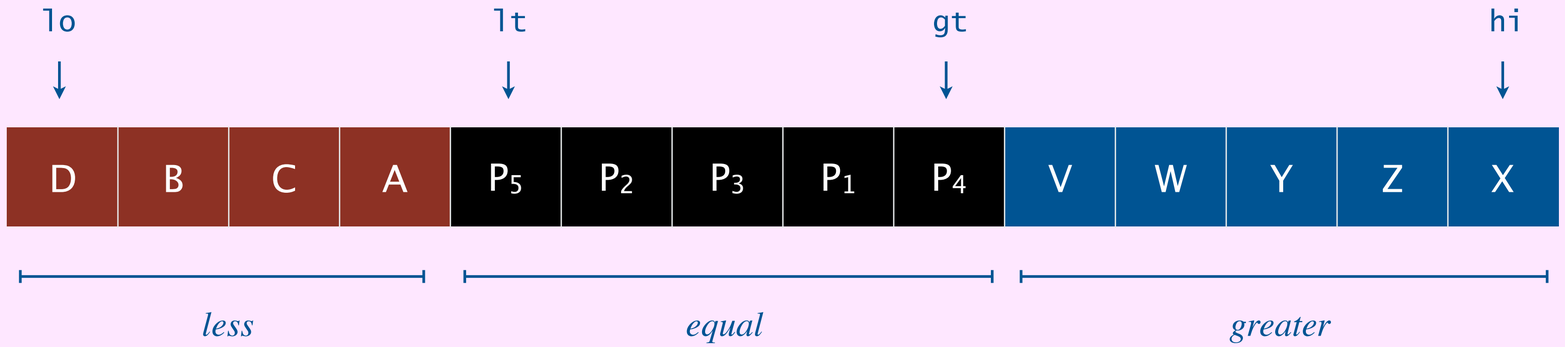
- Let `p = a[lo]` be pivot.
- Scan `i` from left to right and compare `a[i]` to `p`.
  - less: exchange `a[i]` with `a[lt]`; increment both `lt` and `i`
  - greater: exchange `a[i]` with `a[gt]`; decrement `gt`
  - equal: increment `i`



Dijkstra's 3-way partitioning algorithm: demo

- Let `p = a[lo]` be pivot.
- Scan `i` from left to right and compare `a[i]` to `p`.
  - less: exchange `a[i]` with `a[lt]`; increment both `lt` and `i`
  - greater: exchange `a[i]` with `a[gt]`; decrement `gt`
  - equal: increment `i`

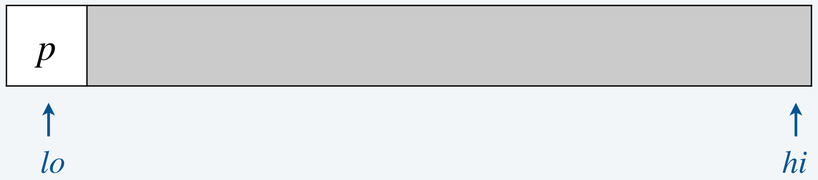
以 p=a[lo] 为基准，用扫描指针 i 从左向右遍历，分三种情况处理：
小于 p 交换 a[i] 与 a[lt] lt++, i++
大于 p 交换 a[i] 与 a[gt] gt--（i 不变，需重新比较交换来的元素）
等于 p 无交换 i++



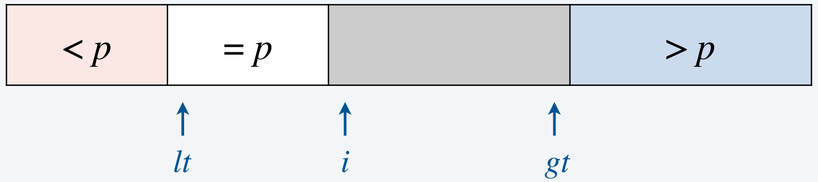
3-way quicksort: Java implementation

```
private static void sort(Comparable[] a, int lo, int hi) {
   if (hi <= lo) return;
   Comparable p = a[lo];

   int lt = lo, gt = hi;
   int i = lo + 1;
   while (i <= gt) {
      int cmp = a[i].compareTo(p);
      if      (cmp < 0) exch(a, lt++, i++);
      else if (cmp > 0) exch(a, i, gt--);
      else              i++;
   }

   sort(a, lo, lt - 1);
   sort(a, gt + 1, hi);
}
```

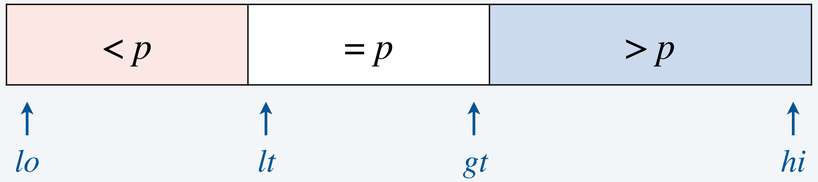
before



during



after



Quicksort: quiz 5

What is the worst-case number of compares to 3-way quicksort an array of length n containing only 7 distinct values?

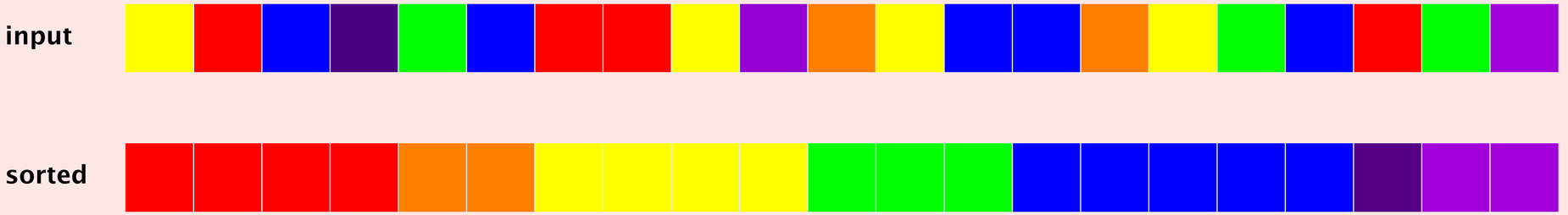
A. $\Theta(n)$

B. $\Theta(n \log n)$

C. $\Theta(n^2)$

D. $\Theta(n^7)$

每次用3-way，都可以划分成三个区间，然后在左右子区间中分别再进行3-way
所以k种不同值，最多进行k次3-way（最坏情况比如，每次只有=p和>p，也就是递归处理的时候没有拆分成左右两个子问题）（在最坏情况下，仍然有每次都能确定一个位置这个基本情况），因此最多处理七次就可以，每次处理不超过n个元素，所以是n级别的
每次partition只需要n的开销，所以选A



Sorting summary

	inplace?	stable?	best	average	worst	remarks
selection	✔		$\frac{1}{2}\ n^2$	$\frac{1}{2}\ n^2$	$\frac{1}{2}\ n^2$	*n exchanges*
insertion	✔	✔	n	$\frac{1}{4}\ n^2$	$\frac{1}{2}\ n^2$	*use for small n or partially sorted arrays*
merge		✔	$\frac{1}{2}\ n \log_2 n$	$n \log_2 n$	$n \log_2 n$	$\Theta(n \log n)$ *guarantee; stable*
timsort		✔	n	$n \log_2 n$	$n \log_2 n$	*improves mergesort when pre-existing order*
quick	✔		$n \log_2 n$	$2\ n \ln n$	$\frac{1}{2}\ n^2$	$\Theta(n \log n)$ *probabilistic guarantee; fastest in practice*
3-way quick	✔		n	$2\ n \ln n$	$\frac{1}{2}\ n^2$	*improves quicksort when duplicate keys*
?	✔	✔	n	$n \log_2 n$	$n \log_2 n$	*holy sorting grail*

number of compares to sort an array of n elements

https://algs4.cs.princeton.edu

2.3 QUICKSORT

- *quicksort*
- *selection*
- *duplicate keys*
- *system sorts*

Sorting applications

Sorting algorithms are essential in a broad variety of applications:

- Sort a list of names.
- Organize an MP3 library.
- Display Google PageRank results.
- List RSS feed in reverse chronological order.

← *obvious applications*

- Find the median.
- Identify statistical outliers.
- Binary search in a database.
- Find duplicates in a mailing list.

← *problems become easy once items are in sorted order*

- Data compression.
- Computer graphics.
- Computational biology.
- Load balancing on a parallel computer.

. . .

← *non-obvious applications*

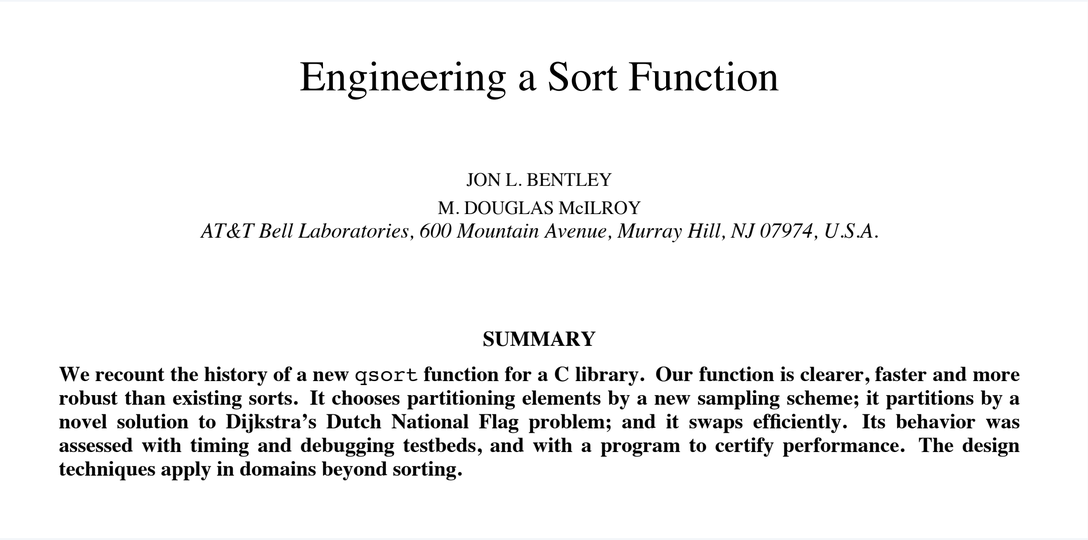
Engineering a system sort (in 1990s)

Bentley–McIlroy quicksort.

- Cutoff to insertion sort for small subarrays.
- Pivot selection: median of 3 or Tukey's ninther.
- Partitioning scheme: Bentley–McIlroy 3-way partitioning.

sample 9 items

similar to Dijkstra 3-way partitioning
(but fewer exchanges when not many equal keys)



Engineering a Sort Function

JON L. BENTLEY
M. DOUGLAS McILROY
AT&T Bell Laboratories, 600 Mountain Avenue, Murray Hill, NJ 07974, U.S.A.

SUMMARY

We recount the history of a new `qsort` function for a C library. Our function is clearer, faster and more robust than existing sorts. It chooses partitioning elements by a new sampling scheme; it partitions by a novel solution to Dijkstra's Dutch National Flag problem; and it swaps efficiently. Its behavior was assessed with timing and debugging testbeds, and with a program to certify performance. The design techniques apply in domains beyond sorting.

In the wild. C, C++, Java 6, ….

A Java mailing list post (Yaroslavskiy, September 2009)

Replacement of quicksort in java.util.Arrays with new dual-pivot quicksort

Hello All,

I'd like to share with you new Dual-Pivot Quicksort which is faster than the known implementations (theoretically and experimental). I'd like to propose to replace the JDK's Quicksort implementation by new one.

...

The new Dual-Pivot Quicksort uses *two* pivots elements in this manner:

1. Pick an elements P1, P2, called pivots from the array.
2. Assume that P1 <= P2, otherwise swap it.
3. Reorder the array into three parts: those less than the smaller pivot, those larger than the larger pivot, and in between are those elements between (or equal to) the two pivots.
4. Recursively sort the sub-arrays.

The invariant of the Dual-Pivot Quicksort is:

[< P1 | P1 <= & <= P2 } > P2]

...

https://mail.openjdk.java.net/pipermail/core-libs-dev/2009-September/002630.html

Another Java mailing list post (Yaroslavskiy–Bloch–Bentley)

Replacement of quicksort in java.util.Arrays with new dual-pivot quicksort

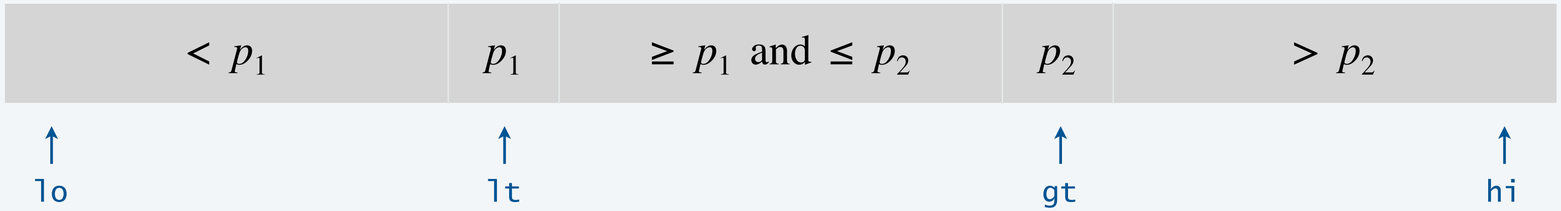
```
Date: Thu, 29 Oct 2009 11:19:39 +0000
Subject: Replace quicksort in java.util.Arrays with dual-pivot implementation

Changeset: b05abb410c52
Author:    alanb
Date:      2009-10-29 11:18 +0000
URL:       http://hg.openjdk.java.net/jdk7/tl/jdk/rev/b05abb410c52

6880672: Replace quicksort in java.util.Arrays with dual-pivot implementation
Reviewed-by: jjb
Contributed-by: vladimir.yaroslavskiy at sun.com, joshua.bloch at google.com,
jbentley at avaya.com

! src/share/classes/java/util/Arrays.java
+ src/share/classes/java/util/DualPivotQuicksort.java
```

https://mail.openjdk.java.net/pipermail/compiler-dev/2009-October.txt

Dual-pivot quicksort

Use two pivots p_1 and p_2 and partition into three subarrays:

- Keys less than p_1.
- Keys between p_1 and p_2.
- Keys greater than p_2.



Recursively sort three subarrays (skip middle subarray if $p_1 = p_2$).

degenerates to Dijkstra's 3-way partitioning

In the wild. Java 8, Java 11, Python unstable sort, Android, …

System sort

Premise. Suppose you are the lead architect of a new programming language.

Q. Which sorting algorithm(s) would you use for the system sort? Defend your answer.

System sorts in Java 8 and Java 11

`Arrays.sort()` and `Arrays.parallelSort()`.

- Has one method for `Comparable` objects.
- Has an overloaded method for each primitive type.
- Has an overloaded method for use with a `Comparator`.
- Has overloaded methods for sorting subarrays.

Algorithms.

- Timsort for reference types.
- Dual-pivot quicksort for primitive types.
- Parallel mergesort for `Arrays.parallelSort()`.

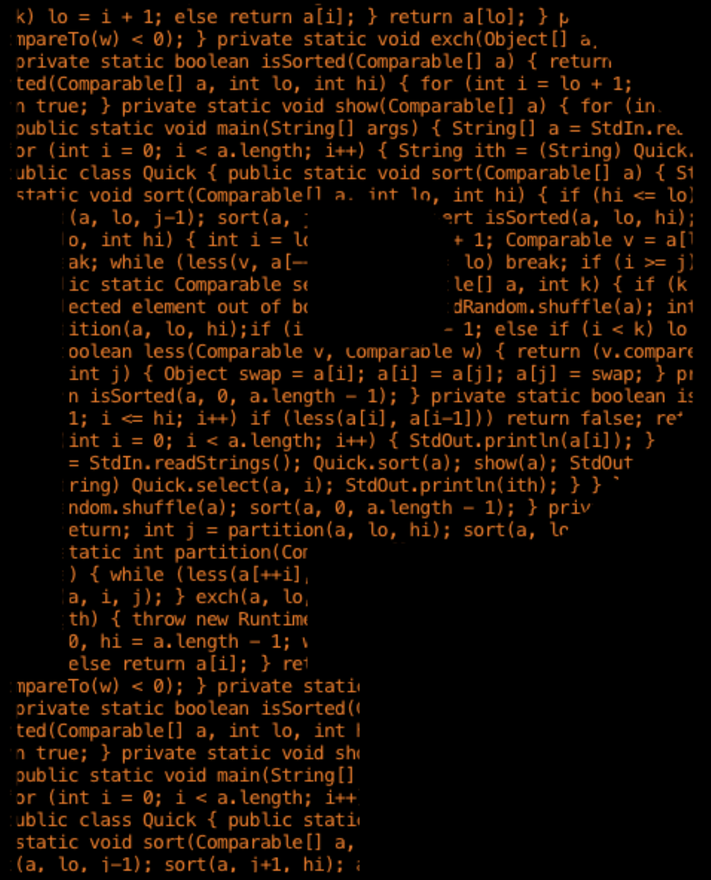
Q. Why use different algorithms for primitive and reference types?

Bottom line. Use the system sort!

Credits

image	source	license
C.A.R. Hoare	Wikimedia	CC BY-SA 2.0 FR
Bob Sedgewick	sedgewick.io	by author
Music of Quicksort	Brad F. Lyon	
Coin Toss	clipground.com	CC BY 4.0
Apocalypse Network Skin	istyles.com	
Harmonic Integral	Wikimedia	public domain
Programmer Icon	Jaime Botero	public domain
Dutch National Flag	Adobe Stock	Education License
CS@Princeton T-Shirt	Ruth Dannenfelser *20	

A final thought



cs @ Princeton